

Waypoint Logistics Platform — Tech Triathlon 2026 Spec

Sep 25, 2026 · @Nimesha Periyapperuma

Summary

The plan is feasible in 10 days, but only if we trim it: ship one responsive web app (PWA) for all four roles, use REST, keep it a modular monolith, and drop Flutter and agentic AI from the core build.

We're building a delivery planning system for Waypoint Group: 120 outlets across three brands (Fresh 80, Style 25, Tech 15), 60 vehicles, two depots (Peliyagoda, Kandy). It must connect ordering, planning, loading, delivery and receipt across four roles: Dispatcher, Loader, Driver and Store Manager.

Headline recommendations:

- **Web first, not mobile first.** The booklet requires a responsive web app and judges test driver and loader flows on phone-sized browser screens. Native apps are optional extras, so Flutter is a second codebase that earns almost no marks.
- **Offline is scored, not optional.** Degradation, offline operation and recovery is 10% of the Hackathon score. A PWA with IndexedDB and a sync queue covers it.
- **The allocation engine is the heart.** Planning and allocation is 20%. It must respect capacity, reefer, van-only, mall windows, time budgets and fuel quotas, and record deferral reasons.
- **Engineering quality is the biggest single criterion (25%).** Clean modules, docs, tests on the engine and a working `docker compose up` matter more than extra features.
- **The real deadline is Day 5, not Day 10.** The Designathon (due Tue 29 Sep, 11:59 PM) becomes our implementation spec, and fidelity to it is scored. Design only what we can build.
- **Auditing is a core module.** Every state change writes an append-only, hash-chained audit event in the same transaction, with mandatory reasons for deferrals, overrides and failed deliveries.
- **Drivers work from a local database.** The phone holds the day's route in IndexedDB and queues actions until signal returns.

- **Infrastructure costs nothing.** Compose for judges; a free-tier autoscaled Kubernetes cluster for the live URL, with Prometheus, Loki and Grafana.

Key dates (Sri Lanka time): Designathon Tue 29 Sep · Hackathon Sun 4 Oct, 11:59 PM · Datathon Fri 9 Oct. All three phases are weighted equally.

Challenge requirements

Every item below is a hard requirement from the booklet; anything not listed is our choice.

Workflow the system must support

Stage	Role	What the system must do
Place order	Store manager	Capture and confirm the order before the 4 PM cutoff
Close orders	Dispatcher	Pull confirmed orders into one queue; late orders roll to the next run
Plan and allocate	Dispatcher	Assign orders to vehicles and trips; identify and record deferrals with reasons
Load	Loader	Show load order matching the stop sequence; flag missing or damaged items before departure
Deliver	Driver	Follow the route, record each stop and proof of delivery, including offline
Confirm receipt	Store manager	Confirm what arrived, report issues, see ETA and deferral notices
Plan capacity	Dispatcher	Use demand forecasts to plan vehicles, drivers and reefer capacity

Operating constraints the plan must respect

- **Capacity:** each trip within both weight (kg) and volume (m³) limits.
- **Temperature:** chilled goods only on reefer vehicles (12 reefer trucks + 4 reefer vans = 16).
- **Access:** `van_only` outlets need a van; `mall_dock` outlets only within the mall window.
- **Windows:** each outlet has a delivery window; Fresh generally before 8 AM.

- **Trips:** max 2 trips per vehicle per day; operating Monday to Saturday per `calendar.csv`.
- **Depot:** vehicles only serve outlets of their home depot.
- **Fuel:** weekly quota per vehicle, consumed by route distance.
- **Time budgets (Datathon rules, worth reusing):** Fresh trips 270 min per vehicle (3:30–8:00 AM); Style + Tech 480 min combined. One trip = one brand + one district.

Hackathon deliverables

- Responsive web app covering all four roles; driver and loader judged on phone-sized screens.
- Public deployed URL, kept live through review, semifinal and finale, plus four seeded accounts (one per role).
- GitHub monorepo named `TeamName_SolutionName` with README (setup, seeded accounts, numbered judge walkthrough, departures from the design).
- `docker-compose.yml` and `.env.example` at the root; `docker compose up` starts everything including DB and seed data.
- `docs/` folder with an architecture diagram, data model and AI tool disclosure.
- Seeded with the shared datasets and at least one realistic delivery day where demand exceeds capacity.
- 5–8 minute unlisted YouTube demo: all four roles, then code and architecture.

Hackathon judging weights

Criterion	Weight
Engineering quality and architecture	25%
Functional completeness across all four roles	20%
Planning and allocation engine	20%
Degradation, offline operation and recovery	10%
Fidelity to the Day 5 design	10%
Demo video	10%
Creativity	5%

Clients, roles and permissions

We treat "multi-client" as multiple front-end experiences on one backend, not multi-tenancy. Waypoint is one company; its three brands share one fleet, so brand is a data attribute, not a tenant boundary.

RBAC is role plus scope. A role says what you can do; a scope says which records you can touch.

Role	Device and conditions	Scope	Can do	Cannot do
Dispatcher	Large screen, stable connection, Peliyagoda office	All depots (or one depot)	Close order queue, run and adjust allocation, defer with reason, view live progress, reassign, view forecasts	Record delivery outcomes
Loader	Shared tablet or terminal at the dock, Peliyagoda or Kandy	Own depot	See load lists in reverse stop order, tick items, flag missing or damaged, release vehicle	Change the plan
Driver	Personal phone, patchy coverage, uses it when stopped	Own vehicle and today's trips	Start trip, arrive, record outcome, capture proof of delivery, report issues, work offline	See other vehicles
Store manager	Desktop or phone at the counter	Own outlet	Place and edit orders before cutoff, see status and ETA, see deferral notices, confirm receipt, report issues	See other outlets

Role	Device and conditions	Scope	Can do	Cannot do
Admin (optional)	Web	Everything	Manage users and master data	—

Implementation notes:

- **Library: Better Auth**, self-hosted inside the NestJS API with sessions stored in our Postgres. Wire it in with `@thalless/nestjs-better-auth`, which adds a global guard, `@Session()` and `@Roles()` decorators. It's community maintained and needs Nest's body parser disabled; stay on the Express adapter, since Fastify support is beta.
- **Plugins:** `username` (simple judge logins), `admin` (user roles, handy for seeding and demos), and access control for per-role permissions. Skip the `organization` plugin; we're not multi-tenant.
- **Scope:** add `outletId`, `depotId` and `vehicleId` as extra user fields. Roles gate the route; a Nest guard plus a scope filter in every service query gate the records.
- **Sessions:** cookie sessions on one origin (Caddy serves the web app and routes `/api` to Nest), so no CORS setup and no tokens in localStorage. Add the `bearer` plugin only if a Flutter app comes back.
- **Offline:** give sessions a long sliding expiry (around 7 days) so drivers aren't logged out mid-route. The offline outbox must not depend on a live session: on a 401 after reconnect, prompt re-login, then replay the queue. Never drop queued events.
- **Shared loader tablet:** quick sign-out/sign-in, and a "checked by" name on each load check.
- **Seeding:** create the four judge accounts through Better Auth's server API in the seed script, so passwords are hashed the same way as real sign-ups.
- **Alternatives considered:** Passport + `@nestjs/jwt` is the safe fallback if the Nest integration fights us, but we'd hand-roll refresh and revocation. Keycloak adds a heavy extra container and slows `docker compose up`. Auth0 or Clerk need external keys, which breaks a self-contained fresh install.

Tech stack evaluation

Most of the stack holds up; the two changes that matter are dropping Flutter for a PWA and demoting Google Maps from a core dependency to a display feature.

Area	Proposed	Verdict	Recommendation and reasoning
Web client	React + TypeScript	Keep	Use Vite, React Router, TanStack Query, Tailwind + a component kit (shadcn/ui or Mantine). One app, role-based routes, responsive layouts.
Mobile client (drivers)	Flutter	Drop for the Hackathon	Booklet requires a responsive web app and judges drivers on phone-sized web screens; native is optional. Flutter means a second codebase, second auth and a second offline layer. Build the driver flow as an installable PWA instead. Revisit Flutter only if we finish early.
Web data layer	Relay (GraphQL) / Axios (REST)	Change	With REST: generate a typed client from OpenAPI (orval or openapi-typescript) + TanStack Query for caching and optimistic updates. Relay is heavy and has strict schema conventions; not worth learning in 10 days.
Backend	TypeScript + NestJS	Keep	Modules map cleanly to our domains; shared types with the frontend in a monorepo.
API style	GraphQL or REST or both	REST	See next section.

Area	Proposed	Verdict	Recommendation and reasoning
Maps	Google Maps (front + back)	Optional, display only	Outlets are given by district, not coordinates, and the booklet's time model uses <code>district_travel.csv</code> . Google routing would give different numbers from the rules judges check against, plus billing and a public API key. Use Leaflet + OpenStreetMap tiles with district centroids (or geocoded outlet pins) for the map view; keep planning maths on the supplied tables.
Database	PostgreSQL on Supabase	Keep, as plain Postgres	Managed Postgres saves us DB ops during the build. Nest is the only thing that talks to it, through Prisma or Drizzle on the connection string. Compose still runs a local postgres container so a fresh install works without Supabase. See Supabase notes below.
Queue and cache	Redis	Keep, lightly	Use it for BullMQ jobs (allocation runs, notifications) and Socket.IO adapter if needed. Skip application caching; the dataset is tiny.
Real-time	Not listed	Add	Server-Sent Events or Socket.IO gateway so the dispatcher sees driver updates live and store managers see ETAs change.

Area	Proposed	Verdict	Recommendation and reasoning
File storage	Supabase Storage	Keep, behind an S3 interface	Proof-of-delivery photos and signatures in a private bucket. Talk to it through its S3-compatible API with the AWS SDK, so local Compose can swap in MinIO with only env changes. Nest hands out short-lived signed URLs for viewing.
Offline	Not listed	Add	Service worker (Workbox via vite-plugin-pwa), IndexedDB (Dexie) for an outbox of driver and loader actions, idempotent sync endpoint.
Containers	Docker Compose	Keep	Services: <code>web</code> , <code>api</code> , <code>worker</code> , <code>postgres</code> , <code>redis</code> , <code>minio</code> (optional), <code>seed</code> (one-shot), <code>caddy</code> for HTTPS in production.
Logging	?	Add	Prometheus + Grafana for metrics, Loki for logs; the audit trail stays in Postgres. See Observability and Auditing.
Deployment	Not listed	Add	Compose for judges and local work; a free-tier Oracle OKE cluster with HPA and KEDA autoscaling for the public HTTPS URL. See Infrastructure.
Testing	Not listed	Add	Vitest/Jest unit tests on the allocation engine and constraint validator; one Playwright happy-path walkthrough if time allows.
Monorepo	Not listed	Add	pnpm workspaces (or Turborepo): <code>apps/web</code> , <code>apps/api</code> , <code>packages/shared</code> (types, constraint rules, Zod schemas).

Area	Proposed	Verdict	Recommendation and reasoning
Auth	Better Auth	Keep	Self-hosted, TypeScript-first, Postgres sessions, fits Compose with no external service. Use the community NestJS integration on Express; Passport + JWT is the fallback. Details under Clients, roles and permissions.

Supabase notes

Use Supabase as infrastructure (managed Postgres + object storage), not as our backend.

- **No Supabase Auth, no supabase-js in the browser, no reliance on RLS.** Better Auth owns users and Nest enforces RBAC; mixing both gives us two sources of truth for permissions.
- **docker compose up must stay self-contained.** `.env.example` defaults `DATABASE_URL` and `S3_*` to the local `postgres` and `minio` containers; only the deployed env points at Supabase. Self-hosting the full Supabase stack in Compose adds around ten containers and isn't worth it.
- **Connection strings:** pooled (transaction mode) for the running app, direct or session connection for migrations (Prisma `directUrl`, `pgbouncer=true` on the pooled URL).
- **Free tier pauses projects after a week of inactivity** ([Supabase pricing](#)). The deployment must stay live through review, semifinal and finale, so either run a scheduled DB ping from the worker or budget for Pro (\$25/month).
- **Free tier limits:** 500 MB database and 1 GB file storage. Plenty for our data, but compress POD photos on the phone before upload.
- **Realtime:** keep SSE from Nest. Supabase Realtime would need Supabase-issued tokens on every client.
- **Region:** pick Singapore, the closest to Sri Lanka, for both the project and our VPS.

API layer decision

Go REST only, with an OpenAPI spec and generated TypeScript client; do not run both.

Factor	REST + OpenAPI	GraphQL + Relay
Team familiarity	High	Mixed; you flagged it may confuse the team
Optimistic updates	TanStack Query <code>onMutate</code> covers it	Built in, but needs Relay store knowledge
Offline sync	Simple batch endpoint <code>POST /sync</code> with idempotency keys	Mutations queue offline awkwardly; persisted queries add setup
File uploads (proof of delivery)	Multipart, trivial	Needs extra spec or a REST side-channel anyway
Real-time	SSE/WebSocket alongside	Subscriptions need extra server setup
Judges reading the code	Swagger UI at <code>/docs</code> is easy to demo	Schema explorer is also fine
Setup cost in 10 days	Low	High (Relay compiler, Node interface, connections)

Running both doubles the API surface and the tests for no scoring gain.

Conventions:

- Resource-oriented routes: `/orders`, `/plans/{date}`, `/plans/{date}/allocate`, `/trips/{id}/load-checks`, `/stops/{id}/events`, `/sync`.
- Zod or class-validator DTOs; generate OpenAPI via `@nestjs/swagger`, then the web client via `orval`.
- Every write from driver or loader carries a client-generated UUID so replays after reconnect are idempotent.
- Consistent error shape and HTTP status codes; version prefix `/api/v1`.

Architectural paradigms

Yes to a modular monolith, yes to in-process event-driven design, and no to agentic AI in the core flow.

Paradigm	Verdict	Why
Modular monolith	Adopt	One deployable, one database, clear module boundaries. Easy to run with <code>docker compose up</code> , easy to explain in the demo, and scores well on architecture without microservice overhead.
Event-driven	Adopt, in-process	Domain events connect the roles: a plan publish reaches the loader, a delivery event reaches the store manager. Use NestJS EventEmitter for in-process events, BullMQ on Redis for async jobs, and a Postgres outbox table so events are not lost. No Kafka or separate broker.
Agentic AI	Defer; optional stretch only	The allocation must be feasible and explainable, which favours deterministic rules. An LLM in the loop adds latency, cost, an external key in a public deploy and non-reproducible results. Creativity is only 5%.

If we want an AI touch after the core is done, the safest option is a "Explain this plan" panel that turns the engine's structured deferral reasons into a plain-language summary for the dispatcher. It should read from our data and never change the plan. Note that the Datathon bans pre-trained models and proprietary API modelling, so keep any LLM use well away from Datathon work.

Core domain events:

- `OrderPlaced`, `OrderCutoffClosed`
- `PlanPublished`, `OrderDeferred`
- `LoadShortfallFlagged`, `VehicleDispatched`
- `StopArrived`, `StopCompleted`, `StopFailed`, `EtaUpdated`
- `ReceiptConfirmed`, `IssueReported`
- `DeviceSynced`

Architecture, modules and data model

One React PWA talks to one NestJS API; a worker process shares the same codebase and handles background jobs.

```

flowchart LR
    U[React PWA<br/>4 role views] -->|REST + SSE| API[NestJS API]
    U -.offline outbox.-> IDB[(IndexedDB)]
    API --> PG[(PostgreSQL)]
    API --> R[(Redis)]
    R --> W[Worker<br/>BullMQ jobs]
    W --> PG
    API --> S3[(MinIO<br/>POD photos)]
    C[Caddy HTTPS] --> U
    C --> API
  
```

Caddy fronts both apps; the worker runs allocation and notification jobs off Redis.

Backend modules

Module	Owns	Key responsibilities
Identity	users, roles, scopes	Login, JWT, guards
Master data	outlets, vehicles, depots, calendar, district travel, service allowance	CSV import and seed, read APIs
Ordering	orders, order lines	Place, edit, cutoff at 4 PM, confirmation
Planning	plans, trips, assignments, deferrals	Allocation engine, constraint validator, manual overrides, deferral reasons
Fleet	fuel ledger, vehicle status	Weekly quota tracking, workshop status
Loading	load checks, shortfalls	Load list by reverse stop order, flag missing or damaged
Execution	stops, stop events, POD	Arrive, deliver, fail, proof of delivery, ETA recalculation

Module	Owns	Key responsibilities
Receipt	receipts, issues	Confirm receipt, report issues
Sync	device outbox, idempotency keys	Batch replay, conflict resolution
Notifications	notifications	In-app feed and SSE fan-out to each role
Audit	audit log	Who changed what and why
Forecasting (stretch)	forecasts	Show Datathon forecasts or a simple baseline for capacity planning

Core data model

Entity	Key fields
<code>outlet</code>	id, brand, district, depot, dock_type, parking_constraint, mall_window, window_open, window_close
<code>vehicle</code>	id, type, temp, weight_cap_kg, volume_cap_m3, km_per_l, weekly_fuel_quota_l, depot, status
<code>calendar_day</code>	date, is_operating, is_payday, festival, festival_ramp, monsoon, iso_week
<code>order</code>	id, outlet_id, delivery_date, temp_requirement, units, weight_kg, volume_m3, status, placed_at, deferred_count
<code>plan</code>	id, date, depot, status (draft, published), created_by
<code>trip</code>	id, plan_id, vehicle_id, trip_no (1 or 2), brand, district, planned_minutes, planned_km, status
<code>stop</code>	id, trip_id, order_id, seq, planned_arrival, eta, status
<code>deferral</code>	id, order_id, plan_id, reason_code, note, decided_by, next_date

Entity	Key fields
load_check	id, trip_id, order_id, status (ok, missing, damaged), note, checked_by
stop_event	id, stop_id, type, occurred_at (device time), received_at, client_uuid, payload
proof_of_delivery	id, stop_id, photo_key, signature_key, receiver_name
receipt	id, order_id, status, issues, confirmed_by
fuel_ledger	id, vehicle_id, iso_week, km, litres
audit_log	id, actor, action, entity, before, after, at

Order status moves through a small state machine:

```
stateDiagram-v2
    [*] --> Placed
    Placed --> Confirmed: cutoff closes
    Confirmed --> Planned
    Confirmed --> Deferred
    Deferred --> Confirmed: next run
    Planned --> Loaded
    Loaded --> OutForDelivery
    OutForDelivery --> Delivered
    OutForDelivery --> Failed
    Delivered --> Received
    Delivered --> Disputed
```

Allocation engine (outline)

A greedy, explainable heuristic is enough; aim for feasible and well-reasoned, not optimal.

1. Load confirmed orders for the date and depot; load available vehicles and remaining fuel.
2. Group orders by brand + district (one trip = one brand, one district).
3. Score priority: deferred yesterday, days since last served, chilled Fresh, tight or mall windows, order age.
4. Pack each group first-fit decreasing into compatible vehicles: reefer for chilled, van for van_only , weight and volume caps.

5. Compute trip minutes from `district_travel.csv` and `service_allowance.csv`; check Fresh 270 min and Style + Tech 480 min budgets, max 2 trips, and fuel left this week.
6. Anything that doesn't fit becomes a deferral with a reason code (no reefer capacity, van shortage, time budget, fuel quota, window conflict).
7. Dispatcher reviews, drags orders between trips, and the validator rejects any move that breaks a rule, saying which rule.

Share the constraint validator in `packages/shared` so the UI can show violations instantly and the same code can check the Datathon Task 2B file.

Auditing

Auditing is a core module, not a log file: every state change in the system writes an audit event in the same database transaction as the change, so nothing can happen without a record.

This answers three of the booklet's problems directly: deferrals lack a clear record, disputes depend on memory, and changes reach people through untracked phone calls.

Rules

1. **Same transaction.** The domain service and `AuditService.record()` share one Prisma/Drizzle transaction. If the audit write fails, the change rolls back.
2. **Append-only.** The app's DB role has only `INSERT` and `SELECT` on `audit_event`. A trigger raises an error on `UPDATE`, `DELETE` and `TRUNCATE`.
3. **Tamper-evident.** Each row stores `prev_hash` and `hash = sha256(prev_hash + canonical row)`. A nightly worker job re-walks the chain and raises an alert if it breaks; the daily head hash is also written to object storage.
4. **Reasons are mandatory** for decisions people will question later. The UI can't submit them without a reason code, and the API rejects them too.
5. **Two clocks.** Store device time and server receive time, in UTC, shown in Asia/Colombo. Offline events synced late are flagged, not reordered.
6. **Idempotent.** Replayed offline actions are deduplicated by `client_uuid`, so a flaky connection can't create duplicate audit rows.

Audit event schema

Field	Purpose
<code>id</code> , <code>seq</code>	UUID plus a monotonic sequence for ordering
<code>actor_id</code> , <code>actor_role</code> , <code>device_id</code>	Who did it, in which role, on which device (important for the shared loader tablet)
<code>source</code>	<code>web</code> , <code>pwa</code> , <code>offline_sync</code> , <code>engine</code> , <code>system</code>
<code>action</code>	e.g. <code>order.placed</code> , <code>plan.published</code> , <code>order.deferred</code> , <code>stop.completed</code>
<code>entity_type</code> , <code>entity_id</code>	What changed
<code>before</code> , <code>after</code>	JSONB snapshots or a diff of the changed fields
<code>reason_code</code> , <code>reason_note</code>	Why (mandatory for the actions below)
<code>occurred_at</code> , <code>recorded_at</code>	Device time and server time
<code>correlation_id</code> , <code>client_uuid</code>	Trace one request or one offline action end to end
<code>prev_hash</code> , <code>hash</code>	Tamper-evidence chain

What gets audited

Role	Actions audited	Reason required for
Store manager	Order placed, edited, cancelled; receipt confirmed; issue reported	Cancellation, dispute or issue
Dispatcher	Cutoff closed, allocation run, plan edited, plan published, order deferred, trip reassigned, override of a validator warning	Every deferral, every manual override, any edit after publish
Allocation engine	Each run: engine version, input snapshot hash, parameters, served and deferred counts	Each deferral carries a machine reason code (reefer capacity, van shortage, time budget, fuel quota, window conflict)
Loader	Item checked, missing, damaged; vehicle released	Missing or damaged items

Role	Actions audited	Reason required for
Driver	Trip started, arrived, delivered, failed, POD captured, issue reported, sync completed	Failed delivery or partial delivery
System	Login, failed login, logout, role or scope change, seed and import runs, chain verification result	Role or scope change

Where people see it

- **Order timeline:** every step from placed to received, with who, when, device and reason. This is the dispute screen.
- **Outlet deferral history:** consecutive skips highlighted so the dispatcher doesn't skip the same outlet twice in a row.
- **Dispatcher audit view:** filter by date, role, action, outlet or vehicle; CSV export.
- **Plan explainer:** the engine's reason codes per deferred order, straight from audit data.

Show the order timeline in the demo video; it proves the four roles are connected and makes the architecture tangible.

Driver local database

The driver's phone keeps a local IndexedDB database (via Dexie) holding the whole day's route, so a driver who loses signal in the Kandy corridor can finish every stop and sync later.

The PWA's service worker caches the app shell, so it also opens with no connection. The loader tablet uses the same pattern for load checks.

What lives on the phone

Local table	Contents	Direction
<code>trips</code>	Today's trips for this vehicle, trip status	Server → phone
<code>stops</code>	Stop sequence, outlet name, address, dock type, delivery window, planned ETA	Server → phone

Local table	Contents	Direction
<code>order_lines</code>	What to hand over at each stop	Server → phone
<code>outbox</code>	Queued actions: arrived, delivered, failed, issue, each with <code>client_uuid</code> and device time	Phone → server
<code>pod_blobs</code>	Compressed POD photos and signatures waiting to upload	Phone → server
<code>sync_state</code>	Last successful sync, server version, pending count	Local only

How it works

1. **Pre-load before departure.** When the loader releases the vehicle, the phone downloads the full day and marks it "ready offline". The UI shows this clearly.
2. **Write local first.** Every driver action updates the local stop and appends to the outbox, even when online. The screen never waits on the network.
3. **Background sync.** When online, the outbox flushes in order to `POST /sync` in batches. Photos upload separately and link by `client_uuid`.
4. **Idempotent server.** The server ignores any `client_uuid` it has already applied, so retries are safe.
5. **Conflict rules.** Plan data is server-wins (the dispatcher may have changed the route); stop events are append-only and never overwritten. If a synced event hits a stop the dispatcher reassigned, it's accepted, audited and flagged for the dispatcher.
6. **Pull changes.** After pushing, the phone pulls anything newer than its last version (reassignments, cancelled stops) and shows a banner.
7. **Visible state.** A badge shows online or offline and "3 actions waiting to sync". Nothing is silently lost.

Degradation screen candidate

"Driver offline mid-route" fits here: the driver completes stops offline, the dispatcher sees the vehicle's last known position and "no signal since 06:12", and the store manager sees a stale ETA with a warning. When signal returns, events sync, the audit trail shows device time versus sync time, and ETAs update.

Request persistent storage (`navigator.storage.persist()`) so the browser doesn't evict the database under storage pressure.

Observability

Use the Grafana stack: Prometheus for metrics, Loki for logs, Grafana to view both. Prometheus doesn't store logs, so it needs Loki alongside it.

```
flowchart LR
    API[NestJS API] -->|/metrics| P[Prometheus]
    W[Worker] -->|/metrics| P
    API -->|JSON stdout| A[Grafana Alloy]
    W -->|JSON stdout| A
    A --> L[Loki]
    P --> G[Grafana]
    L --> G
    P --> AM[Alertmanager]
```

Apps export metrics and write JSON logs to stdout; Alloy ships logs to Loki, and Grafana shows both on one dashboard.

Layer	Tool	What it captures
Logs	nestjs-pino → stdout → Grafana Alloy → Loki	Request ID, user ID, role, route, latency, errors; labels for app, env, pod
Metrics	prom-client at /metrics → Prometheus	HTTP rate, latency, errors; BullMQ queue depth; sync batch size and lag; allocation run time; orders deferred per run
Dashboards and alerts	Grafana + Alertmanager	API error rate, queue backlog, drivers not synced for 30+ min, audit chain check failure
Correlation	Request ID middleware; client_uuid on offline actions	Jump from a Grafana log line to the matching audit event
Health	@nestjs/terminus at /health (DB, Redis, storage)	Kubernetes liveness and readiness probes; Compose healthchecks
Client errors (optional)	Sentry free tier	Frontend and sync failures on phones

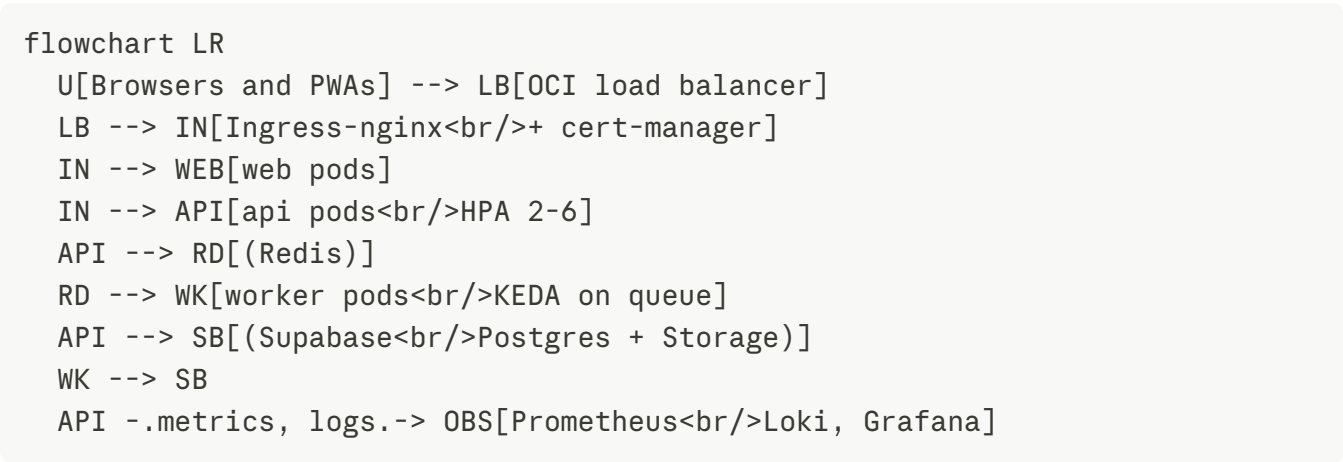
Logs are for engineers and expire; the audit trail is the business record and doesn't. Never treat Loki as the audit store.

In Kubernetes, install it with `kube-prometheus-stack` and the Loki Helm chart. In local Compose, keep observability behind a profile (`docker compose --profile observability up`) so the default start stays light for judges.

Infrastructure

Run two environments from the same images: Docker Compose for judges and local work (required by the booklet), and a free-tier autoscaled Kubernetes cluster for the public URL.

Our real load is small (120 outlets, 60 drivers, a few dispatchers), with bursts at the 3:30–8:00 AM Fresh window and at the 4 PM cutoff. The cluster shows we've designed for scale without paying for it.



The cluster holds only stateless pods and Redis; Postgres and file storage live on Supabase.

Cluster

Item	Choice	Notes
Provider	Oracle Cloud (OKE) on Always Free Ampere A1 nodes	OKE doesn't charge for cluster management, only for worker nodes (Galovics)

Item	Choice	Notes
Free capacity	2 OCPU / 12 GB total, e.g. 2 nodes × 1 OCPU / 6 GB	Oracle quietly halved the free A1 allowance from 4 OCPU / 24 GB in 2026 (TerminalBytes); size to the new limit
Images	Multi-arch (amd64 + arm64) via docker buildx , pushed to GitHub Container Registry	A1 nodes are ARM
Ingress	ingress-nginx + cert-manager (Let's Encrypt) behind the free OCI load balancer	HTTPS is required for the PWA and service worker
Fallback	k3s on a single VM, or Compose on the same VM	If OCI setup or "out of capacity" errors eat more than half a day, switch immediately

Workloads

Workload	Kind	Scaling	Notes
web	Deployment (nginx serving the Vite build)	2 replicas	Static; cheap
api	Deployment	HPA: CPU 70%, min 2, max 6	Stateless; readiness and liveness probes on /health
worker	Deployment	KEDA on BullMQ queue length, min 1, max 4	Allocation runs, notifications, audit chain check
redis	StatefulSet (or Upstash free tier)	1 replica	Queues and SSE pub/sub
observability	kube-prometheus-stack + Loki Helm charts	—	Keep retention short to fit free nodes

Workload	Kind	Scaling	Notes
Postgres, file storage	Supabase (external)	Managed	Keeps the cluster stateless

Autoscaling details

- **Pods:** `metrics-server` + HPA for the API; KEDA's Redis scaler for workers.
- **Nodes:** the free quota caps us at 2 OCPU, so node autoscaling (OKE cluster autoscaler) is documented but only switched on with a paid node pool. Say this plainly in the architecture doc.
- **SSE across replicas:** each API pod subscribes to Redis pub/sub, so an event raised on one pod reaches clients connected to any pod. No sticky sessions needed.
- **Safety:** resource requests and limits on every pod, PodDisruptionBudget on `api`, rolling updates.

Delivery pipeline

- GitHub Actions: lint, test (including allocation engine tests), build multi-arch images, push to GHCR, then `helm upgrade` (or `kustomize`) to the cluster.
- Manifests live in `deploy/k8s` (base + overlays); the Compose file stays at the repo root as required.
- Secrets as Kubernetes Secrets from GitHub Actions secrets; `.env.example` documents every variable.

Cost

Item	Cost
OKE control plane + A1 nodes + load balancer	Free (Always Free)
Supabase	Free, or \$25/month Pro to avoid pausing
GHCR, GitHub Actions	Free for our usage
Prometheus, Loki, Grafana	Free, self-hosted
Domain	Optional; a free <code>sslip.io</code> -style hostname works

Feasibility and 10-day plan

Feasible with the trimmed scope; not feasible with Flutter, GraphQL + Relay, Google routing and agentic AI all in.

The real constraint is that design and build overlap: the Designathon is due on Day 5, and the Datathon runs in parallel until Day 15. Backend work that doesn't depend on screens (schema, seed, auth, allocation engine) should start on Day 1 while designers work.

Proposed item	Effort	Fits in 10 days?
React PWA, four role views	High	Yes, core
NestJS REST + Postgres + seed	Medium	Yes, core
Allocation engine + validator	High	Yes, core; start Day 1
Offline driver and loader flows	Medium	Yes, if scoped to driver stop events and load checks
Real-time updates (SSE)	Low	Yes
Redis + BullMQ	Low	Yes
Docker Compose + VPS deploy	Medium	Yes; deploy by Day 7, not Day 10
Map view (Leaflet)	Low	Yes, nice-to-have

Proposed item	Effort	Fits in 10 days?
Google Maps routing	Medium	Not worth it; conflicts with the supplied travel tables
GraphQL + Relay	High	No
Flutter driver app	High	No; stretch after Day 10 only
Agentic AI	Medium–High	No; small optional “explain plan” only
Audit module (hash chain, reasons, timeline view)	Medium	Yes, core; build with the schema on Day 1–2 so every module uses it from the start
Kubernetes on OKE with HPA/KEDA	Medium–High	Yes if one person owns it from Day 2; fall back to Compose on a VM by Day 7 if not live
Prometheus + Loki + Grafana	Low–Medium	Yes via Helm charts; one dashboard is enough

Day-by-day plan

Day	Date	Design track	Build track
1	Fri 25 Sep	Read datasets, personas, pick degradation scenario	Monorepo, Compose, Postgres schema, CSV import
2	Sat 26 Sep	Screen flows for all four roles	Auth + RBAC, seed four accounts, ordering APIs
3	Sun 27 Sep	Hi-fi screens, degradation screen	Allocation engine v1 + unit tests
4	Mon 28 Sep	Prototype links, rationale per screen	Planning APIs, deferral reasons, fuel ledger
5	Tue 29 Sep	Demo video, AI disclosure; submit by 11:59 PM	Web shell, auth, routing, design tokens from the submitted design
6	Wed 30 Sep	—	Dispatcher: order queue, run allocation, edit plan, deferrals

Day	Date	Design track	Build track
7	Thu 1 Oct	—	Loader view, load checks, shortfall flags; first VPS deploy
8	Fri 2 Oct	—	Driver PWA, offline outbox, sync, POD photo
9	Sat 3 Oct	—	Store manager: orders, ETA, deferral notices, receipt; SSE live updates
10	Sun 4 Oct	—	Seed demo day, README walkthrough, docs, demo video; freeze by 8 PM

Suggested split for a team of four to five

- **Backend lead:** schema, seed, auth, APIs, deployment.
- **Engine owner:** allocation engine and validator (also reusable for Datathon Task 2B).
- **Frontend 1:** dispatcher and store manager views.
- **Frontend 2:** driver and loader PWA, offline sync.
- **Design + Datathon:** Designathon deliverables to Day 5, then Datathon from Day 6.

Gaps, risks and open questions

The original requirements covered the stack but missed most of what judges will actually test: offline sync, deferral records, deployment and the seeded walkthrough.

Missed in requirements gathering

- **Offline sync design:** outbox in IndexedDB, client UUIDs, idempotent `/sync`, conflict rules (server wins for plan data, append-only for stop events), visible "pending sync" badges.
- **Deferral handling:** reason codes, consecutive-skip flag per outlet, notice to the store manager, and deferred orders auto-queued for the next run.
- **Proof of delivery:** photo, receiver name, optional signature, stored with timestamps.
- **Loader shortfall flow:** what happens to the plan and store manager when an item is missing or damaged before departure.

- **ETA:** planned arrival from the trip time model, updated from driver events; shown to store managers.
- **Order cutoff and time zone:** 4 PM cutoff in Asia/Colombo; a demo clock override so judges can test outside real time.
- **Fuel quota:** weekly ledger per vehicle, using `km_per_1` and trip distance.
- **Degradation screen:** pick one scenario now (e.g. driver offline in the Kandy corridor, or a reefer breaks down at 4 AM) and build exactly that.
- **Deployment:** public HTTPS URL, kept live through the finale; domain, VPS and cost owner.
- **Seed data:** shared datasets plus one realistic overloaded delivery day that makes the walkthrough work on a fresh install.
- **Documentation:** README walkthrough, `docs/` architecture diagram and data model, AI tool disclosure, departures from the design.
- **Capacity planning screen:** the workflow lists it; a simple forecast view can satisfy it without Datathon integration.
- **Accessibility and phone UX:** large touch targets, used when safely stopped, works in sunlight.

Risks

Risk	Impact	Mitigation
Design promises more than we can build	Loses fidelity marks (10%)	Design only the scoped screens; log departures in README
Allocation engine late or buggy	Hits the 20% engine criterion and the demo	Start Day 1, unit-test against the booklet's worked examples (101 min Gampaha trip, 213 of 270 min)
Offline sync edge cases	Hits the 10% degradation criterion	Keep offline scope to driver stop events and loader checks
Deployment left to Day 10	No public URL, no marks	Deploy by Day 7 and redeploy daily

Risk	Impact	Mitigation
Datathon starves the build team	Both phases suffer	Fix people per track now
Dataset terms of use	Disqualification	Keep the repo private until allowed, or confirm with organisers whether seeded CSVs may sit in the repo
Kubernetes setup eats build time, or OCI has no free ARM capacity	No live URL, lost days	Timebox to half a day; same images run on Compose on one VM as the fallback
Audit writes bolted on late	Gaps in the record, weaker dispute and deferral story	Make AuditService part of the shared transaction helper so no service can skip it

Open questions

- ☐ Team size and who owns each track?
- ☐ Do we keep Flutter as a post-Hackathon stretch, or drop it entirely?
- ☐ Is the repo allowed to be public given the data confidentiality terms? (Ask tech-triathlon@rootcode.io.)
- ☐ Which degradation scenario do we design and build?
- ☐ Where do we host (VPS provider) and who pays?
- ☐ Should the dispatcher view cover both depots or one at a time?